

NAME: AVANCHA KIRAN KUMAR

REGNO: 20233093

SEC: A_2

1. Assume that a system has a 32-bit virtual address with a 4-KB page size.

Write a C program

that is passed a virtual address (in decimal) on the command line and have it

output the page

number and offset for the given address. As an example, your program would run

as

follows:

`./a.out 19986`

Your program would output:

The address 19986 contains:

page number = 4

offset = 3602

Writing this program will require using the appropriate data type to store 32 bits.

We

encourage you to use unsigned data types as well.

Assignment - 9

Name: Avancha Kiran
RegNo: 20231093

Q1) #include <stdio.h>
#include <stdlib.h>
~~#include <st~~

int main (int argc, char* argv[]) {

if (argc != 2) {

printf("Usage: %s <virtual address>\n", argv[0]);

return 1;

}

uint32_t virtual_address =

uint32_t (atoi(argv[1]));

const uint32_t PAGE_SIZE = 4096;

uint32_t page_number = virtual_address / PAGE_SIZE;

uint32_t offset = virtual_address % PAGE_SIZE;

printf("The address %u contains:\n", virtual_address);

printf("Page number = %u\n", page_number);

printf("offset = %u\n", offset);

return 0;

Output:

```
bharath@ubuntu:~/Documents/osAssignment-9$ gcc q1.c
bharath@ubuntu:~/Documents/osAssignment-9$ ./a.out 19986
The address 19986 contains:
page number = 4
offset = 3602
bharath@ubuntu:~/Documents/osAssignment-9$
```

2. Write a program that implements the FIFO and LRU page-replacement algorithms. First, generate a random page reference string where page numbers range from 0 to 9. Apply the random page reference string to each algorithm, and record the number of page faults incurred by each algorithm. Implement the replacement algorithms so that the number of page frames can vary from 1 to 7. Assume that demand paging is used.

Name: Avincha Khan Kumer
Regno: 2023093

```
Q2) #include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>
#define MAX_PAGE_FRAMES 7
#define MIN_PAGE_FRAMES 1
#define MAX_PAGE_NUMBER 9
#define REFERENCE_STRING_LENGTH 100

void generateReferenceString (int *referenceString,
                             int length) {
    for (int i = 0; i < length; i++) {
        referenceString[i] = rand() % (MAX_PAGE_NUMBER);
    }
}

bool pageExists (int *frameArray, int numFrame, int page) {
    for (int i = 0; i < numFrame; i++) {
        if (frameArray[i] == page) {
            return true;
        }
    }
    return false;
}
```



```

int findOldestPosition (int* frameArray, int numFrames,
    int* ages, int* currentTime) {
    int oldest = 0;
    for (int i = 1; i < numFrames; i++) {
        if (ages[i] < ages[oldest]) {
            oldest = i;
        }
    }
    return oldest;
}

```

```

int findLRUPosition (int* frameArray, int numFrames,
    int* lastUsed) {
    int lruPos = 0;
    for (int i = 1; i < numFrames; i++) {
        if (lastUsed[i] < lastUsed[lruPos]) {
            lruPos = i;
        }
    }
    return lruPos;
}

```

```

int fifo (int* referenceString, int length, int numFrames) {
    int frameArray [MAX-PAGE-FRAMES];
    int ages [MAX-PAGE-FRAMES];
    int pageFaults = 0;
    int currentTime = 0;
}

```


Name: Avanche Kiran Kumar

RegNo: 20233093

```
for(int i=0; i<numFrames; i++) {  
    *frameArray[i] = -1;  
    ages[i] = -1;  
}
```

```
{  
    for(int i=0; i<length; i++) {  
        int page = referenceString[i];  
        if(!pageExists(frameArray, numFrames, page)) {  
            pageFaults++;  
            int emptyFramePos = -1;  
            for(int j=0; j<numFrames; j++) {  
                if(frameArray[j] == -1) {  
                    emptyFramePos = j;  
                    break;  
                }  
            }  
            if(emptyFramePos != -1) {  
                frameArray[emptyFramePos] = page;  
                ages[emptyFramePos] = currentTime++;  
            }  
        }  
    }  
}
```


Avancharan Kumar

20233093

else {

int oldestpos

= findOldestPosition (frameArray, numFrames,
ages, & currentTime);

frameArray [oldestpos] = page;

ages [oldestpos] = currentTime + 1;

}

return page faults;

}

int lru (int *referenceString, int length, int numFrames) {

int frameArray [MAX-PAGE-FRAMES];

int lastUsed [MAX-PAGE-FRAMES];

int pageFaults = 0;

int currentTime = 0;

for (int i = 0; i < numFrames; i++) {

frameArray [i] = -1;

lastUsed [i] = -1;

}

Avanchar Kiran Kumar

20232013

20/10/23

```
for (int i=0; i<length; i++) {
```

```
    int page = referenceString[i];
```

```
    currentTime++;
```

```
    bool exists = false;
```

```
    for (int j=0; j<numFrames; j++) {
```

```
        if (frameArray[j] == page) {
```

```
            lastUsed[j] = currentTime;
```

```
            exists = true;
```

```
            break;
```

```
        }  
        if (!exists) {
```

```
            pageFaults++;
```

```
            int emptyFramePos = -1;
```

```
            for (int j=0; j<numFrames; j++) {
```

```
                if (frameArray[j] == -1) {
```

```
                    emptyFramePos = j;
```

```
                    break;
```

```
                }  
            }  
        }
```


Avanchar Kishan Kumar
20233093

```
if (emptyFramePos != -1) {
    frameArray[emptyFramePos] = page;
    lastUsed[emptyFramePos] = currentTime;
}
else {
    int lruPos = findLRUPosition(frameArray,
                                numFrames, lastUsed);
    frameArray[lruPos] = page;
    lastUsed[lruPos] = currentTime;
}
}
}
return pageFaults;
}

int main() {
    srand(time(NULL));
    int referenceString[REFERENCE_STRING_LENGTH];
    generateReferenceString(referenceString,
                            REFERENCE_STRING_LENGTH);
    printf("Generated page Reference string: \n");
}
```


AvanCha Kiran Kumar
20233093

```
for (int i=0; i< REFERENCE_STRING_LENGTH; i++) {  
    printf("%d", referenceString[i]);  
    if ((i+1)%20==0) printf("\n");
```

```
}  
printf("\n\n");
```

```
printf("Number of page faults for different  
algorithms and frame counts: \n");
```

```
printf("frames | FIFO | LRU\n");
```

```
for (int frames = MIN_PAGE_FRAMES; frames <= MAX_PAGE_FRAMES; frames++) {
```

```
    int fifoFaults = fifo(referenceString,  
        REFERENCE_STRING_LENGTH, frames);
```

```
    int lruFaults = lru(referenceString, REFERENCE-  
        STRING_LENGTH, frames);
```

```
    printf("%d | %d | %d\n", frames, fifoFaults,  
        lruFaults);
```

```
    return 0;
```


Output:

```
bharath@ubuntu:~/Documents/osAssignment-9$ gcc q2.c
```

```
bharath@ubuntu:~/Documents/osAssignment-9$ ./a.out
```

Generated Page Reference String:

```
4 9 0 0 0 5 2 7 7 3 9 5 9 7 3 0 2 8 8 2
3 4 6 3 6 7 4 5 7 6 8 1 6 9 1 8 4 5 6 3
0 5 0 9 2 3 9 7 3 9 9 8 4 5 4 0 4 8 7 1
6 7 4 2 6 8 1 2 3 9 7 4 4 7 5 8 1 5 7 6
4 6 5 8 4 9 0 0 9 9 4 5 7 8 0 5 6 3 0 2
```

Number of page faults for different algorithms and frame counts:

Frames	FIFO	LRU
1	92	92
2	84	83
3	71	73
4	62	58
5	56	54
6	41	44
7	28	36